

C++ 프로그래밍

참조 (Reference)

참조란?

****기존 변수에 붙이는 또 다른 이름(별명)****이다. 포인터처럼 주소를 다루지만, 문법이 훨씬 단순하고 안전하다.

```
1  int x = 42;
2
3  int* p = &x;    // 포인터: x의 주소를 저장
4  int& r = x;     // 참조: x의 별명(alias)
5
6  printf("%d\n", x);    // 42
7  printf("%d\n", *p);   // 42 ← 역참조 연산자 필요
8  printf("%d\n", r);    // 42 ← 그냥 변수처럼 사용
9
10 r = 100;          // x를 직접 수정 (역참조 불필요)
11 printf("%d\n", x);   // 100
```

`r` 은 새로운 변수가 아니다. `r` 과 `x` 는 **동일한 메모리**를 가리키는 두 개의 이름이다.

참조의 규칙

참조는 포인터와 달리 엄격한 제약이 있다. 이 제약 덕분에 포인터보다 안전하다.

반드시 선언 시 초기화

```
int x = 10;

int& r = x;    // ✅ 선언과 동시에 초기화 필수

int& r2;        // ❌ 컴파일 오류 - 무엇의 별명인지 불명확
```

재대입(rebind) 불가

```
int a = 10, b = 20;
int& r = a;    // r은 a의 별명

r = b;         // ⚠️ r이 b를 가리키도록 바뀌는 게 아님!
              //   a에 b의 값(20)을 대입하는 것

printf("%d %d\n", a, r); // 20 20
```

null 참조 없음

```
int* p = nullptr;    // ✅ 포인터는 null 가능
int& r = ???;         // ❌ null 참조는 존재하지 않음

// nullptr 역참조는 런타임 크래시
// *p = 10; // 🚬 UB (Undefined Behavior)

// 참조는 항상 유효한 변수를 가리키도록 강제됨
```

참조는 선언 이후 별명이 바뀌지 않는다

```
int a = 10, b = 20;
int& r = a;    // r은 a의 별명 - 영원히

r = b;        // a = b 와 동일 (rebind 아님)
printf("%d\n", a); // 20
printf("%d\n", r); // 20 (r은 여전히 a)
```

참조 vs 포인터: 기본 비교

포인터 (C/C++)

```
int x = 10;
int y = 20;

int* p = &x;    // x를 가리킴

*p = 100;        // 역참조로 값 변경 (x = 100)

p = &y;           // ✅ 다른 변수를 가리키도록 재지정 가능

int* q = nullptr; // ✅ null 가능
// *q = 5;         // ❌ 크래시

printf("%d\n", *p); // 20
```

참조 (C++)

```
int x = 10;
int y = 20;

int& r = x;      // x의 별명

r = 100;         // 그냥 대입 (x = 100)

// r = y;        // ⚠️ x = y 로 해석됨 (rebind 아님)

// int& s;        // ❌ 초기화 없이 선언 불가
// null 참조 없음 - 항상 유효

printf("%d\n", r); // 100
```

참조 vs 포인터: 비교표

항목	포인터 (<code>int* p</code>)	참조 (<code>int& r</code>)
초기화	선언 후 대입 가능	선언 시 필수
null 가능 여부	<code>nullptr</code> 가능	불가
재지정 (rebind)	<code>p = &y</code> 가능	불가 — 항상 같은 변수
값 접근 문법	<code>*p</code> (역참조 필요)	<code>r</code> (변수처럼 직접 사용)
주소 접근	<code>p</code> 자체가 주소	<code>&r</code> (원본의 주소 반환)
포인터 연산	<code>p++</code> , <code>p + n</code> 가능	불가
배열 원소 순회	✅ 포인터 연산으로 가능	불가
안전성	낮음 (null, 댕글링 위험)	높음

규칙: 재지정이나 null이 필요하면 포인터, 그 외엔 **참조**를 쓰는 것이 C++ 관례다.

함수 인자 전달: C 포인터 vs C++ 참조

함수에서 원본 변수를 수정하려면 C는 **포인터**, C++는 **참조**를 사용한다.

C 스타일 — 포인터 전달

```
// 인자로 주소를 받음
void swap(int* a, int* b) {
    int tmp = *a;
    *a = *b;      // 역참조 필요
    *b = tmp;
}

int main() {
    int x = 10, y = 20;
    swap(&x, &y); // & 연산자로 주소 전달
    printf("%d %d\n", x, y); // 20 10
}
```

- 호출 측에서 `&` 필요
- 함수 내부에서 `*` 역참조 필요
- `nullptr` 전달 가능 → 방어 코드 필요

C++ 스타일 — 참조 전달

```
// 인자로 참조를 받음
void swap(int& a, int& b) {
    int tmp = a;
    a = b;      // 역참조 불필요
    b = tmp;
}

int main() {
    int x = 10, y = 20;
    swap(x, y); // & 없이 그냥 전달
    printf("%d %d\n", x, y); // 20 10
}
```

- 호출 측에서 `&` 불필요 — 자연스러운 문법
- 함수 내부에서 `*` 불필요
- `null` 전달 불가 → 방어 코드 불필요

const 참조

읽기 전용으로 원본을 참조한다. 복사 없이 안전하게 전달하는 가장 일반적인 패턴이다.

const 참조의 특징

```
int x = 42;
const int& r = x;

printf("%d\n", r); // ✅ 읽기 가능
// r = 100;        // ❌ 컴파일 오류 - 쓰기 불가

// 리터럴과 임시 값도 바인딩 가능
const int& cr = 100; // ✅ (일반 참조는 불가)
const double& dr = 3; // ✅ int → double 변환 후 바인딩
```

포인터와 비교

```
// const 포인터 조합은 혼란스러움
const int* p1 = &x; // 가리키는 값이 const
int* const p2 = &x; // 포인터 자체가 const

// const 참조는 단순함
const int& r = x; // 참조로 수정 불가 (값은 변할 수 있음)
```

함수 인자로 const 참조 — 권장 패턴

```
struct Student {
    char name[32];
    int age;
    double gpa;
};

// ❌ 값 전달 - 구조체 전체 복사 발생
void print_v(Student s) {
    printf("%s\n", s.name);
}

// ✅ const 참조 전달 - 복사 없음, 수정도 불가
void print_r(const Student& s) {
    printf("%s\n", s.name);
}

// ✅ 참조 전달 - 복사 없음, 수정 가능
void birthday(Student& s) {
    s.age += 1;
}
```

크기가 큰 객체를 읽기 전용으로 전달할 때는 `const T&` 가 정석이다.

참조 반환

함수가 참조를 반환하면 호출 측에서 직접 대입할 수 있다.

참조 반환 기본

```
int g_value = 10;

// 전역/정적 변수의 참조 반환 - 안전
int& get_value() {
    return g_value;
}

int main() {
    get_value() = 99;           // 직접 대입 가능
    printf("%d\n", g_value);    // 99

    int& r = get_value();       // 참조로 받기
    r = 42;
    printf("%d\n", g_value);    // 42
}
```

❌ 지역 변수 참조 반환 — 절대 금지

```
int& danger() {
    int local = 10;
    return local;    // ❌ 댕글링 참조!
    // local은 함수 종료 시 소멸
    // 반환된 참조는 소멸된 메모리를 가리킴
}

int main() {
    int& r = danger();
    r = 99;    // ❌ UB (Undefined Behavior)
}
```

규칙: 함수의 참조 반환은 매개변수로 받은 참조 또는 전역/정적 변수만 반환해야 한다.

참조와 포인터: 언제 무엇을 쓸까?

참조를 선택하는 경우

```
// ✅ 함수 인자 - 수정이 필요할 때
void increment(int& n) { n++; }

// ✅ 함수 인자 - 크기가 큰 객체를 읽을 때
void print(const std::string& s) {
    printf("%s\n", s.c_str());
}

// ✅ range-for 에서 원본 수정
int arr[] = {1, 2, 3, 4, 5};
for (int& v : arr) {
    v *= 2;    // 원본 배열 수정
}

// ✅ 긴 이름의 객체에 짧은 별명
auto& item = very_long_container_name[idx];
item.value = 42;
```

포인터를 선택하는 경우

```
// ✅ null 가능성이 있을 때
void process(int* p) {
    if (p == nullptr) return;    // null 체크
    *p = 10;
}

// ✅ 재지정이 필요할 때 (다른 대상으로 변경)
int a = 1, b = 2;
int* p = &a;
p = &b;    // 가리키는 대상 변경

// ✅ 배열/포인터 연산이 필요할 때
int arr[] = {1, 2, 3};
int* p = arr;
for (int i = 0; i < 3; i++) {
    printf("%d\n", *(p + i));
}

// ✅ 동적 메모리 관리
int* buf = new int[100];
delete[] buf;
```

요약

```
int x = 10;

int* p = &x;    // 포인터: x의 주소 저장, 역참조(*p)로 접근
int& r = x;     // 참조: x의 별명, 변수처럼 직접 접근
```

상황

권장

함수에서 원본 수정

`void f(int& x)` — 참조

크기가 큰 객체를 읽기 전용으로 전달

`void f(const T& x)` — const 참조

null일 수도 있는 인자

`void f(int* p)` — 포인터

함수 내에서 다른 대상으로 바꿔야 할 때

`int* p` — 포인터

배열 순회·포인터 연산

`int* p` — 포인터

동적 메모리

`new` / `delete` + 포인터

C++에서 포인터는 꼭 필요할 때만 쓰고, 그 외 원본 접근이 필요한 대부분의 경우엔 참조를 사용하는 것이 관례다.